Program: Code on the disk, does nothing until launched

Process: Program in execution, own memory/resources (PCB)

Thread: Smallest unit of CPU work, shares memory but has it's own stack & registers

CPU: Brain of the computer, executes instructions

Each core: one worker that runs 1 thread at a time

More cores = more parallel tasks

Context switching:

- Occurs when threads > cores
- OS scheduler saves current thread state & loads another
- Gives illusion of multi-tasking
- Each core handles it's own switching independently
- Octa-core runs 8 threads at once; rest wait & get swapped

Concurrency: Multi-tasks make program (interleaving) - can happen even on 1 core

Parallelism: Tasks run at the same time - requires multiple cores.

Parallelism ≠ automatically faster, both need careful design.

Race conditions:

- Threads share memory = powerful but dangerous

Example,

counter += 1 is not atomic

- Without locks, results are wrong
- Fix = synchronization

Example,


```
for (i=0; i<1000; i++)
    counter ++
```

Lets say T_0 was switched but T_1 wasn't. So, T_1 incremented counter but it needed to incremented from where T_0 left off & not from 0. T_0 comes back finds not 0 by T_1's incremeted counter.

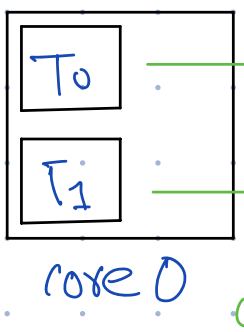
Hyperthreading / Simultaneous multi-threading (SMT)

- One physical core behaves as two logical cores
- OS sees double cores

Example,

4 cores → 8 cores

- Typically, 15-30% performance gains
- Threads share cache & pipelines



→ goes off to handle I/O

→ Can utilize CPU resources

- This way we don't need to save thread state & waste time loading a thread.

Pros:

- Utilizes idle CPU cycles better
- Helps multi-tasking & mixed workloads

Cons:

- Doesn't double performance
- Threads compete for shared resources

NOTE:

Apple doesn't make use of SMT. It instead uses efficiency and performance cores.